



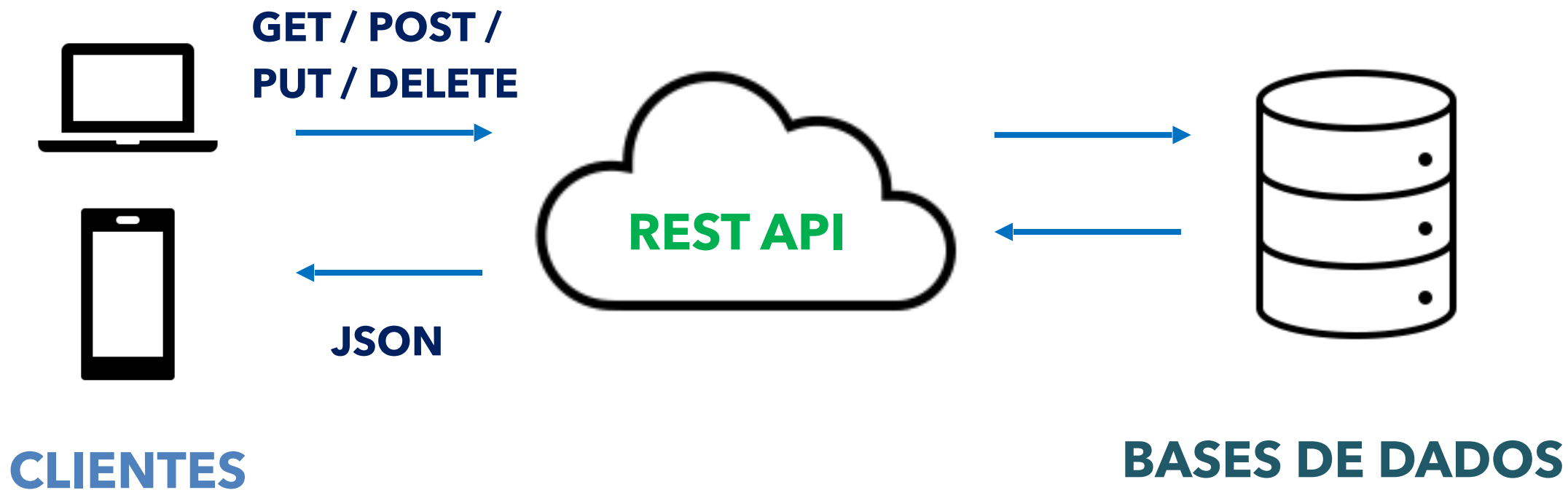
9. API Rest e Integração com o Banco

SCC0219 – Introdução ao Desenvolvimento Web

Profª. Drª. Bruna C. Rodrigues da Cunha

brunaru@icmc.usp.br

Revisão

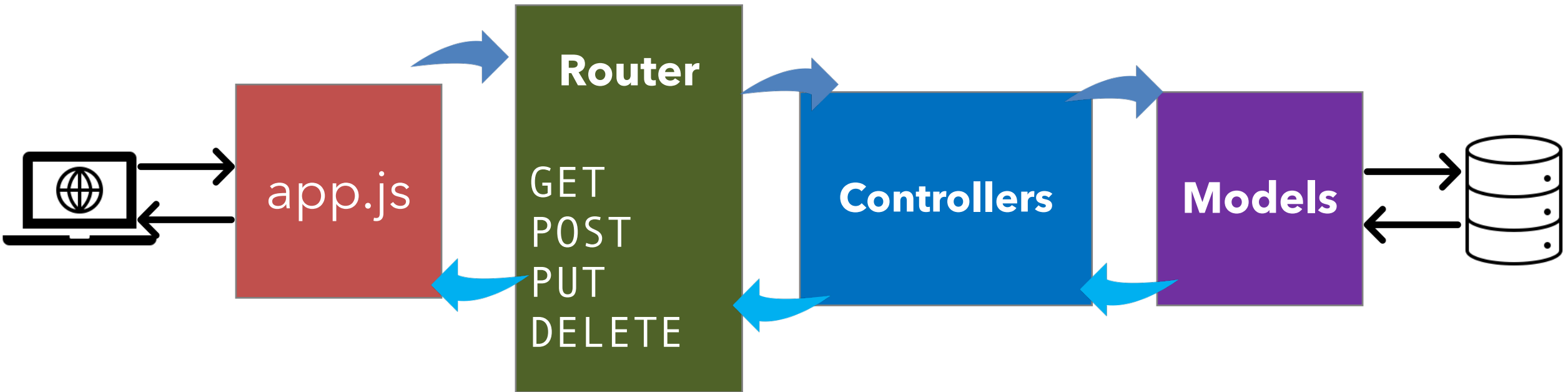


REpresentational **S**tate **T**ransfer

Revisão

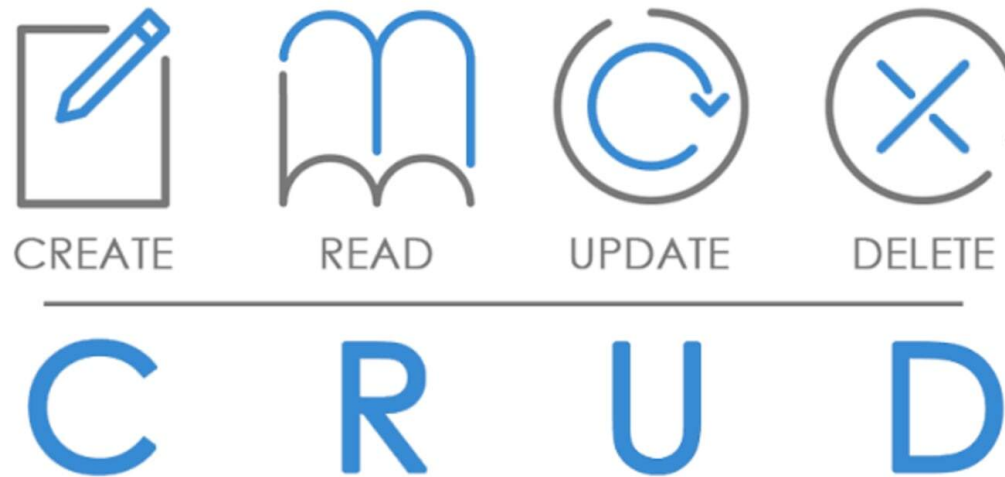
- Express é um *framework* de aplicações web para o Node.js que fornece uma maneira conveniente e flexível de criar e gerenciar aplicações web
 - O Express fornece várias funcionalidades, como gerenciamento de rotas, tratamento de erros e recursos de segurança
 - **Auxilia na criação de APIs REST**
 - Instalação: **npm i express**
-

MVC no Node.js



Revisão

- Na aula anterior utilizamos o Express para criar uma API Rest para representar um recurso **DogBreed**.
- Mas um CRUD sem Banco de Dados não é nada!



| Documentação do Express

Documentação completa da API:

<https://expressjs.com/en/5x/api.html>

Integrando com o Banco de Dados

- Podemos utilizar o módulo do banco e fazer operações diretamente em nosso código.
 - Porém essa alternativa não é eficiente nem adequada para todos os projetos.
 - Exemplos completos em: <https://blog.logrocket.com/crud-rest-api-node-js-express-postgresql/>
-

Exemplo de Conexão com Módulo pg

```
const Pool = require('pg').Pool
```

```
const pool = new Pool({ user: 'me', host: 'localhost', database:  
'api', password: 'password', port: 5432, })
```

Exemplo de Conexão com Módulo pg

```
const getUsers = (request, response) => {  
  pool.query('SELECT * FROM users ORDER BY id ASC', (error, results) => {  
    if (error) {  
      throw error  
    }  
    response.status(200).json(results.rows)  
  })  
}
```

Mapeamento Objeto-Relacional

- **Mapeamento Objeto-Relacional (MOR)** ou **Object Relational Mapper (ORM)** é uma técnica de programação que permite relacionar objetos com dados em um banco de dados.
 - Desta forma, **não é preciso se preocupar com a escrita de código SQL**, dado que a técnica realiza o mapeamento dos objetos em tabelas e dados.
 - Existem diferentes ferramentas de Mapeamento OR. As ferramentas dependem da linguagem de programação utilizada.
 - A forma que o mapeamento é configurado depende da ferramenta.
 - **A ferramenta ORM mapeia o código ao banco automaticamente por meio dessas configurações.**
-

Mapeamento Objeto-Relacional

- Podemos utilizar diferentes ferramentas ORM para o Node.js.
- No caso de bancos relacionais, a **Sequelize** é uma das soluções mais famosas.



- A Sequelize funciona com Postgres, MySQL, MariaDB, SQLite, DB2, Microsoft SQL Server, Snowflake, Oracle DB e Db2 IBM i.
-

Modelagem de Dados de Objeto

- Para bancos não relacionais!
- Para exemplo, para o MongoDB temos o Mongoose

mongoose

elegant **mongodb** object modeling for **node.js**

Let's face it, **writing MongoDB validation, casting and business logic boilerplate is a drag**. That's why we wrote Mongoose.

```
const mongoose = require('mongoose');
mongoose.connect('mongodb://127.0.0.1:27017/test');

const Cat = mongoose.model('Cat', { name: String });

const kitty = new Cat({ name: 'Zildjian' });
kitty.save().then(() => console.log('meow'));
```

Mapeamento Objeto-Relacional

- Para mapear nosso código com o banco de dados utilizando a **Sequelize**, nós temos de instalar o módulo sequelize e o módulo do banco utilizado.
 - Após inicializar um objeto de conexão, é necessário criar os modelos das entidades que devem persistir no banco.
-

Inicialização do Banco com Sequelize

- Instalar os seguintes módulos:

`npm i express`

`npm i sequelize`

`npm i pg`

`npm i cors`

Banco no PostgreSQL: Banco Local

- Para esse exemplo você precisa criar um usuário e um banco.
- Seguem os comandos de criação:

```
psql -d postgres
```

```
CREATE ROLE devweb WITH LOGIN PASSWORD '123456';
```

```
ALTER ROLE devweb CREATEDB;
```

```
\q
```

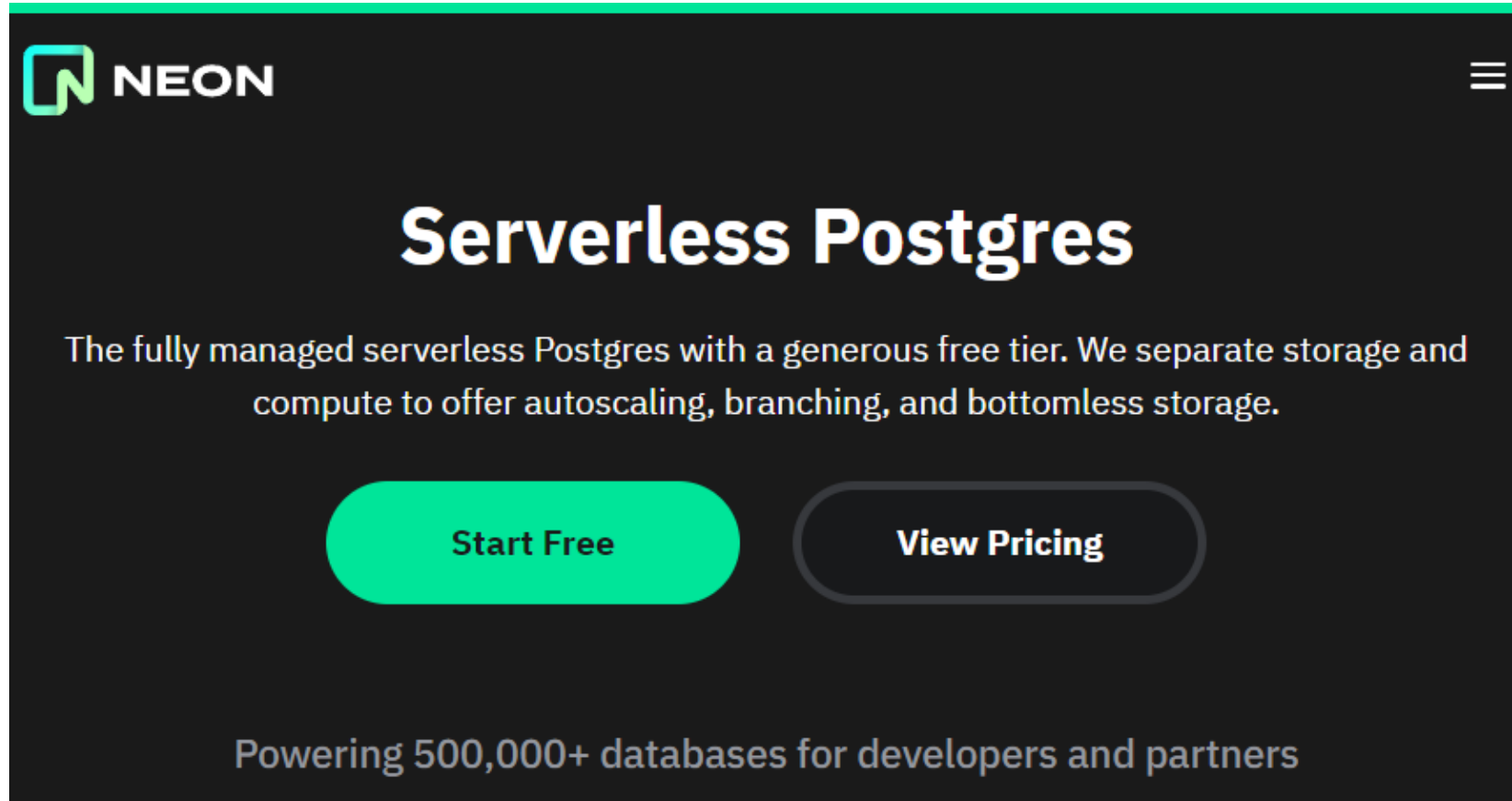
```
psql -d postgres -U devweb
```

```
CREATE DATABASE devapi;
```



PostgreSQL

Banco no PostgreSQL: Banco Serverless



The screenshot shows the Neon website's hero section. At the top left is the Neon logo, consisting of a green square with a white 'N' and the word 'NEON' in white. At the top right is a white hamburger menu icon. The main heading is 'Serverless Postgres' in a large, bold, white font. Below it is a paragraph in white text: 'The fully managed serverless Postgres with a generous free tier. We separate storage and compute to offer autoscaling, branching, and bottomless storage.' There are two buttons: a solid blue rounded rectangle with the text 'Start Free' in white, and a white rounded rectangle with a blue border and the text 'View Pricing' in blue. At the bottom, in a smaller white font, is the text 'Powering 500,000+ databases for developers and partners'.

NEON

Serverless Postgres

The fully managed serverless Postgres with a generous free tier. We separate storage and compute to offer autoscaling, branching, and bottomless storage.

[Start Free](#) [View Pricing](#)

Powering 500,000+ databases for developers and partners

Inicialização do Banco com Sequelize

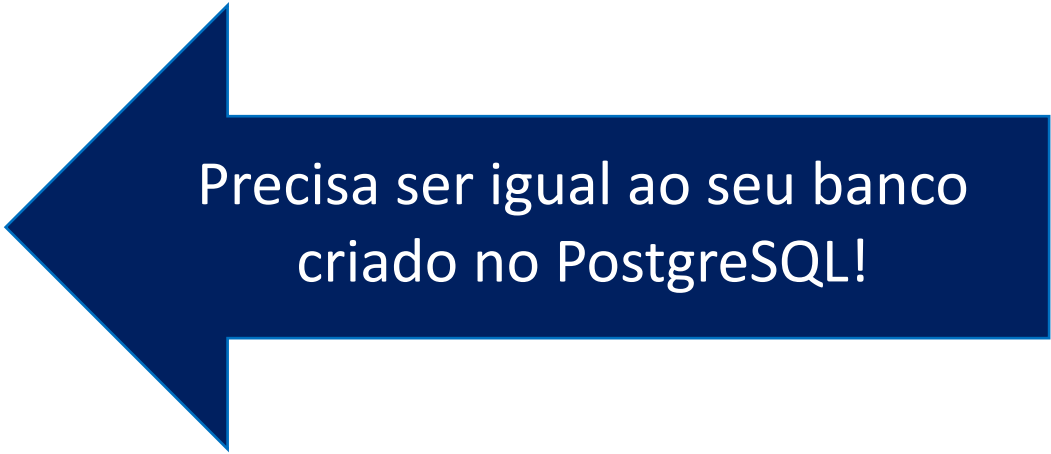
- Neste exemplo vamos utilizar um arquivo **.env** na raiz do projeto com os dados do nosso banco:

DB_NAME=devapi

DB_USER=devweb

DB_PASSWORD=123456

DB_HOST=localhost



Precisa ser igual ao seu banco
criado no PostgreSQL!

Inicialização do Banco Local com Sequelize

models/dbconfig.js

```
import Sequelize from 'sequelize'
import dotenv from 'dotenv'
dotenv.config()
const dbName = process.env.DB_NAME
const dbUser = process.env.DB_USER
const dbHost = process.env.DB_HOST
const dbPassword = process.env.DB_PASSWORD

const sequelize = new Sequelize(dbName, dbUser, dbPassword,
  { dialect: 'postgres', host: dbHost })

export default sequelize
```

Inicialização do Banco Servless com Sequelize

models/dbconfig.js

```
import { Sequelize, Model, DataTypes } from "sequelize"

const PGHOST = process.env['PGHOST']
const PGUSER = process.env['PGUSER']
const PGDATABASE = process.env['PGDATABASE']
const PGPASSWORD = process.env['PGPASSWORD']

const sequelize = new Sequelize(PGDATABASE, PGUSER, PGPASSWORD, {
  dialect: "postgres", host: PGHOST, port: 5432,
  dialectOptions: { ssl: { require: true, rejectUnauthorized:
false } },
})

export default sequelize
```

Criação de um Modelo

models/client.model.js

```
import { Model, DataTypes } from 'sequelize'
import sequelize from './dbconfig.js'

class Client extends Model { }

Client.init({
  id: { type: DataTypes.INTEGER, autoIncrement: true, primaryKey: true },
  name: { type: DataTypes.STRING, allowNull: false },
  document: { type: DataTypes.STRING, allowNull: false }
}, { sequelize: sequelize, timestamps: false })

export default Client
```

Criação de um Modelo

- Existem outras formas de definir o modelo, exemplo:

```
const Client = sequelize.define( 'Client', {  
  id: { type: DataTypes.INTEGER,  
    autoIncrement: true,  
    primaryKey: true },  
  name: { type: DataTypes.STRING,  
    allowNull: false },  
  document: { type: DataTypes.STRING, allowNull: false },  
}, { sequelize: sequelize, timestamps: false })
```

- <https://sequelize.org/docs/v6/core-concepts/model-basics/>

Testando o Modelo

dbsync.js

- Com o modelo de Cliente pronto, já podemos fazer certas ações no banco!

```
import Client from './models/client.model.js'
await Client.sync()
await Client.create({name: "Endeavor", document: "202.002.002-2"})
await Client.findAll().then(function(res) {
  for(let r of res) {
    console.log(r.dataValues)
  })
})
```

- **Observação: as chamadas ao banco são assíncronas!!!**

Promise

- As promessas (*promises*) são a base da programação assíncrona no JavaScript. Uma **promise** é um objeto retornado por uma função assíncrona, que representa o estado atual da operação. No momento em que a promessa é retornada ao “chamador”, o objeto da promessa fornece métodos para lidar com o eventual sucesso ou falha da operação.

```
Client.findAll().then(function(res) {  
    for(let r of res) {  
        console.log(r.dataValues)  
    }  
}).catch(function(e) { console.log(e) })
```

Promise e await

- O operador **await** é usado para aguardar uma **promise** e obter seu valor de cumprimento. Só pode ser usado dentro de uma função assíncrona ou no nível superior de um módulo.

```
await Client.create({name: "Endeavor", document:  
"202.002.002-2"})
```


Controller

controllers/client.controller.js

```
import model from "../models/client.model.js"

function findAll(request, response) {
  model.findAll().then(function (results) {
    response.json(results).status(200)
  }).catch(function (e) { console.log(e) })
}

function findByPk(request, response) {
  model.findByPk(request.params.id).then(function (result) {
    response.json(result).status(200)
  }).catch(function (e) { console.log(e) })
}

function create (request, response) {
  model.create(
    { name: request.body.name, document: request.body.document }
  ).then(function (result) {
    response.status(201).json(result)
  }).catch(function (e) { console.log(e) })
}
```

Controller

controllers/client.controller.js

```
function update(request, response) {
  model.update(
    { name: request.body.name, document: request.body.document },
    { where: { id: request.params.id } }
  ).then(function (result) {
    response.status(200).send()
  }).catch(function (e) { console.log(e) })
}

function deleteByPk(request, response) {
  model.destroy({ where: { id: request.params.id } })
  .then(function (result) {
    if (result == 1) { response.status(200).send() }
    else { response.status(404).send() }
  }).catch(function (e) { console.log(e) })
}

export default { findAll, findByPk, create, update, deleteByPk }
```

Associações

- O Sequelize suporta as associações clássicas de Um-Para-Um, Um-Para-Muitos e Muitos-Para-Muitos.
- Isso é feito com 4 tipos de comandos:
 - **HasOne**
 - **BelongsTo**
 - **HasMany**
 - **BelongsToMany**

Associações

- Para criar uma relação de Um-Para-Muitos (One-To-Many) entre Client e Pet (*i.e.*, um cliente pode ter vários pets), utilizo o comando **hasMany**:

```
Client.hasMany(Pets)
```

- Para criar uma relação sinônima de Muitos-Para-Um (Many-To-One) entre Client e Pet (*i.e.*, um pet pode ser de um cliente), posso utilizar também o comando **belongsTo**:

```
Pet.belongsTo(Client, { foreignKey: 'responsavelId' })
```

- A configuração **foreignKey** permite mudar o nome da chave estrangeira.
-

Associações

models/pet.model.js

```
import { Model, DataTypes } from "sequelize";
import sequelize from "../dbconfig.js";
import Client from "../client.model.js";

class Pet extends Model {}
Pet.init({
  id: { type: DataTypes.INTEGER, autoIncrement: true, primaryKey: true },
  name: { type: DataTypes.STRING, allowNull: false },
  type: { type: DataTypes.STRING, allowNull: false },
  breed: { type: DataTypes.STRING },
  birth: { type: DataTypes.STRING },
}, { sequelize: sequelize, timestamps: false }, );

Pet.belongsTo(Client);
Client.hasMany(Pet);
export default Pet;
```

Pet Controller

controllers/pet.controller.js

```
import model from "../models/pet.model.js"
import client from "../models/client.model.js"

function findAll(request, response) {
  model.findAll().then(function (results) {
    response.json(results).status(200)
  }).catch(function (e) { console.log(e) })
}

function findByPk(request, response) {
  model.findByPk(request.params.id, { include: client }
  ).then(function (result) {
    response.json(result).status(200)
  }).catch(function (e) { console.log(e) })
}
```

Pet Controller

controllers/pet.controller.js

```
function findPetsOfClient(request, response) {  
    model.findAll({ where: { ClientId:  
request.params.id } })  
    ).then(function (results) {  
        response.json(results).status(200)  
    }).catch(function (e) { console.log(e) })  
}
```

Pet Controller

controllers/pet.controller.js

```
function create(request, response) {
  model.create({
    name: request.body.name, type: request.body.type,
    breed: request.body.breed, birth: request.body.birth, ClientId: request.body.ClientId }
  ).then(function (result) {
    response.json(result).status(201)
  }).catch(function (e) { console.log(e) })
}

function deleteByPk(request, response) {
  model.destroy({ where: { id: request.params.id } }
  ).then(function (result) {
    if (result == 1) { response.status(200).send()
    } else { response.status(404).send()
    }
  }).catch(function (e) { console.log(e) })
}
```


Pet Controller

controllers/pet.controller.js

```
function update(request, response) {
  model.update({
    name: request.body.name, type: request.body.type,
    breed: request.body.breed, birth: request.body.birth, ClientId:
request.body.client },
    { where: { id: request.params.id } }
  ).then(function (result) {
    response.json(result).send()
  }).catch(function (e) { console.log(e) })
}

export default {
  findAll, findByPk, create, update, deleteByPk, findPetsOfClient, };
```

Rotas

```
import express from "express";
import breedController from "../controllers/breed.controller.js";
import clientController from "../controllers/client.controller.js";
import petController from "../controllers/pet.controller.js";

const router = express.Router();

router.get("/breeds", breedController.getAll);
router.get("/breeds/:id", breedController.getById);
router.post("/breeds", breedController.create);

router.get("/clients", clientController.findAll);
router.get("/clients/:id", clientController.findByPk);
router.post("/clients", clientController.create);
router.put("/clients/:id", clientController.update);
router.delete("/clients/:id", clientController.deleteByPk);
router.get("/pets", petController.findAll);
router.get("/pets/:id", petController.findByPk);
router.post("/pets", petController.create);
router.put("/pets/:id", petController.update);
router.delete("/pets/:id", petController.deleteByPk);
router.get("/clients/:id/pets", petController.findPetsOfClient);

export default router;
```

| Documentação do Sequelize

<https://sequelize.org/docs/v6/>



Prática

